

Lambda

Lambda 表达式

注意：考虑到算法竞赛的实际情况，本文将不会全面研究语法，只会讲述在算法竞赛中可能会应用到的部分。

本文语法参照 **C++11** 标准，其他高版本的标准语法视情况提及并会特别标注。

Lambda 表达式

Lambda 表达式因数学中的 演算得名，直接对应于其中的 lambda 抽象。编译器在编译时会根据语法生成一个匿名的 [函数对象](#)，以捕获的变量作为其成员，参数和函数体用于实现 `operator()` 重载。

► 函数对象 (Function Object)

以下是 lambda 的一种语法：

```
1 [capture] (parameters) mutable -> return-type {statement}
```

Lambda 表达式本身是一个类，展开后如以下形式：

```
1 class Lambda_1 {
2   private:
3     Lambda_1() : capture-list(init-value) { }
4
5   public:
6     return-type operator()(parameters) const { statement }
7
8   private:
9     mutable capture-list
10 };
```

空的 capture 可以隐式转换为函数指针，例如：

```
1 void (*f)(int, int) = [](int, int) -> void {};
```

下面我们分别对语法中的各部分进行介绍。

statement 函数体

函数体与普通函数函数体类似，除了能访问参数和全局变量等，还可访问 [捕获](#) 的变量。

capture 捕获子句

lambda 以 capture 子句开头，它指定哪些变量被捕获，捕获列表可为空，或指定捕获方式：有 `&` 符号前缀的变量通过 [引用](#) 访问，没有该前缀的变量通过值访问。

我们也可以使用默认捕获模式，捕获 Lambda 中提及的所有变量：`&` 表示捕获到的所有变量都通过引用访问，`=` 表示捕获到的所有变量都通过值访问。

在默认捕获之后，仍然可以为特定的变量 **显式** 指定捕获模式。

如果需要引用访问外部变量 `a`，并通过值访问外部变量 `b`，那么以下捕获子句都可以做到：

- [&a, b]
- [b, &a]
- [&, b]
- [b, &]
- [=, &a]

同时捕获列表也可以用于声明变量，类型由初始化器推导，类似于使用 `auto` 声明变量。

以下是一些常见的例子：

```
1 int a = 0;
2 auto f0 = []() { return a * 9; }; // Error, 无法访问 'a'
3 auto f1 = [a]() { return a * 9; }; // OK, 'a' 被值「捕获」
4 auto f2 = [&a]() { return a++; }; // OK, 'a' 被引用「捕获」
5 auto f3 = [v = a + 1]() {
6     return v + 1;
7 }; // OK, 使用初始化器声明变量 v, 类型与 a 相同
8
9 // 注意, 使用引用捕获时, 请保证被调用时 a 没有被销毁
10 auto b = f2(); // f2 从捕获列表里获得 a 的值, 无需通过参数传入 a
```

generalized capture 带初始化的捕获 (C++14)

自 C++14 起, `capture` 不仅可以用来捕获外部变量, 还可用于声明新的变量并初始化, 例如:

```

1 auto f1 = [val = 520]() {
2     return val;
3 }; // OK, 定义 val 类型为 int, 初始值为 520, 返回值类型 int
4
5 auto f2 = [val = 520LL]() {
6     return val;
7 }; // OK, 定义 val 类型为 long long, 初始值为 520, 返回值类型 long long
8
9 auto f3 = [val = "520"]() {
10     return val;
11 }; // OK, 定义 val 类型为 const char*, 初始值为 "520", 返回值类型 const char*
12
13 auto f4 = [val = "520"s]() {
14     return val;
15 }; // OK, C++14 起, 需要 using namespace std; 或 using namespace std::literals;
16     // 定义 val 类型为 std::string, 初始值为 std::string("520"), 返回值类型
17     // std::string
18
19 auto f5 = [val = std::string("520")]() {
20     return val;
21 }; // OK, 定义 val 类型为 std::string, 初始值为 std::string("520"), 返回值类型
22     // std::string
23
24 auto f6 = [val = std::vector<int>(3, 6)]() {
25     return val;
26 }; // OK, 定义 val 类型为 std::vector<int>, 大小为 3, 元素填充 6, 返回值类型
27     // std::vector<int>
28
29 auto f7 = [val = 520]() -> int {
30     return val;
31 }; // OK, 定义 val 类型为 int, 初始值为 520, 返回值类型 int
32
33 auto f8 = [val = 520]() -> long long {
34     return val;
35 }; // OK, 定义 val 类型为 int, 初始值为 520, 返回值类型 long long

```

定义新的变量不可以省略初始值，变量的类型由初始值的类型决定，相当于：

```
1 auto val = init-value;
```

以下是错误的写法：

```

1 auto f = [val]() { return val; }; // Error: 'val' was not declared in this
2                                 // scope, identifier "val" is undefined

```

初始化值也可以是外部变量，例如：

```

1 int value = 520;
2 auto f = [val = value]() { return val; };
3 std::cout << f(); // Output: 520

```

val 也可以是一个引用类型，可以引用一个外部变量，通过这种方式可以为通过引用捕获的外部变量取个别名，例如：

```

1 int value = 520;
2
3 auto f = [&val = value]() {
4     return val;
5 }; // OK, 定义 val 类型为 int&, 返回值类型 int, 相当于 int& val = value;
6
7 std::cout << f() << '\n'; // Output: 520
8
9 value = 1314;
10
11 std::cout << f() << '\n'; // Output: 1314

```

捕获外部变量和定义新变量可以同时使用。

如果你想在 Lambda 表达式内修改 capture 中定义的新变量，需要使用 `mutable` 关键字，如果是引用则不需要，例如：

```

1 int value = 520;
2
3 {
4     auto f = [val = value]() mutable -> int {
5         return val = 1314;
6     }; // 需要 mutable
7     auto val_f = f();
8     std::cout << value << ' ' << val_f << std::endl; // Output: 520 1314
9 }
10
11 {
12     auto f = [&val = value]() -> int { return val = 1314; }; // 不需要 mutable
13     auto val_f = f();
14     std::cout << value << ' ' << val_f << std::endl; // Output: 1314 1314
15 }

```

详见 [mutable 可变规范](#)。

在 capture 中定义的变量的生命周期跟随 Lambda 表达式的接收方，在以上几个示例中为变量，因为 Lambda 本身其实是一个类，capture 中的所有内容都是这个类的 `private` 成员变量，例如：

```

1 int main() {
2     auto f = [val = 0]() mutable -> int { return ++val; }; // val 被构造和初始化
3
4     std::cout << f() << '\n'; // Output: 1
5     std::cout << f() << '\n'; // Output: 2
6     std::cout << f() << '\n'; // Output: 3
7 } // val 跟随 f 被销毁

```

parameters 参数列表

大多数情况下类似于函数的参数列表，例如：

```

1 int x[] = {5, 1, 7, 6, 1, 4, 2};
2 std::sort(x, x + 7, [](int a, int b) { return (a > b); });
3 for (auto i : x) std::cout << i << " ";

```

这将打印出 `x` 数组从大到小排序后的结果。

由于 **parameters 参数列表** 是可选的，如果不将参数传递给 lambda，并且其声明不包含 [mutable](#)，且没有后置返回值类型，则可以省略空括号。

► 使用 `auto` 声明的参数

显式对象形参 (C++23)

C++23 起, [显式对象形参](#) 可以在 `lambda` 的参数中使用。

```
1 auto nth_fibonacci = [](this auto self, unsigned n) -> unsigned {
2     return n < 2 ? n : self(n - 1) + self(n - 2);
3 };
4
5 cout << nth_fibonacci(10u);
```

mutable 可变规范

使得函数体可以修改通过值捕获的变量。

```
1 int a = 0;
2 auto by_value = [a]() mutable { ++a; };
3 auto by_ref = [&a] { ++a; };
4
5 by_value();
6 by_ref();
```

在执行完 `by_value()` 后, `by_value` 的捕获成员 `a` 为 1, 但外部的变量 `a` 依然为 0。而在执行完 `by_ref()` 后, 外部 `a` 的值变为 1。

return-type 返回类型

用于指定 `lambda` 表达式的返回类型。如果省略, 则返回类型将被自动推断 (行为与用 `auto` 声明返回值的函数一致)。

多个 `return` 语句且推导类型不一致时, 将产生编译错误。

```
1 auto lam = [](int a, int b) -> int { return 0; };
2
3 auto x1 = [](int i) { return i; };
4
5 auto x2 = [](bool condition) {
6     if (condition) return 1;
7     return 1.0;
8 }; // Error, 推导类型不一致
```

泛型 Lambda (C++14)

使用 `auto` 作为参数类型, 可以构造泛型 `lambda`。

```
1 auto add = [](auto a, auto b) { return a + b; };
```

在 [cpp insights](#) 中可以观察到编译器生成的 `lambda` 类定义:

```

1 class add_lambda {
2 public:
3     template <class T, class U>
4     auto operator()(T a, U b) const {
5         return a + b;
6     }
7 };
8
9 add_lambda add{};

```

add 两个参数声明均使用了 auto，对应为 add_lambda 类的 operator() 函数模板的两个模板参数 T 和 U。

Lambda 中的递归

先来看一个编译失败的例子：

```

1 int n = 10;
2
3 auto dfs = [&](int i) -> void {
4     if (i == n)
5         return;
6     else
7         dfs(i + 1); // Error: a variable declared with an auto type specifier
8                     // cannot appear in its own initializer
9 };

```

我们这里尝试在捕获列表中捕获，但是有一个问题，的类型为 auto，要等待等号右边的类型推导完成后才会推导出类型，而 Lambda 要捕获 就必须先确定 的类型后才能创建它的引用变量，好，这会陷入了一个套娃过程。

怎么解决这个问题呢？

1. 显式指定 的类型，可以使用 std::function 替代。

▼ 修改如上代码为：

```

1 int n = 10;
2
3 std::function<void(int)> dfs = [&](int i) -> void {
4     if (i == n)
5         return;
6     else
7         dfs(i + 1); // OK
8 };
9
10 dfs(1);

```

► 不建议使用 [std::function](#) 实现的递归

1. 不通过捕获的方式获取，而是通过函数传参的方式。

▼ 修改如上代码为：

```

1 int n = 10;
2
3 // 参数列表中有参数类型为 auto, 则这个 Lambda 类中的 operator()
4 // 函数将被定义为模板函数, 模板函数可以在稍后被调用时再进行实例化
5 auto dfs = [&](auto& self,
6             int i) -> void // [&] 只会捕获用到的变量, 所以不会捕获 auto dfs
7 {
8     if (i == n)
9         return;
10    else
11        self(self, i + 1); // OK
12 };
13
14 dfs(dfs, 1);

```

▼ auto self、auto& self 和 auto&& self 的区别：

auto& self 和 auto&& self 理论上都会使用 个字节（指针的大小）用作传参，不会发生其他的拷贝。具体要看编译器对 Lambda 的实现方式和对应的优化。而使用 auto self 会发生对象拷贝，拷贝的大小取决于捕获列表中的元素，因为它们都是这个 Lambda 类中的私有成员变量。

1. 可以通过手动展开 Lambda 类，或使用类似写法，这样可以直接声明 的类型。

▼ 修改如上代码为：

```

1 int n = 10;
2
3 class Lambda_1 {
4 public:
5     auto operator()(int i) const -> void {
6         if (i == n)
7             return;
8         else
9             (*this)(i + 1); // OK
10    }
11
12    explicit Lambda_1(int& __n) : n(__n) {}
13
14 private:
15     int& n;
16 } dfs(n);
17
18 dfs(1);

```

1. 如果 lambda 没有捕获任何变量，我们也可以利用函数指针。

如果 lambda 没有捕获任何变量，那么它可以隐式转换为函数指针。同时 lambda 此时也可以声明为 static，函数指针类型也可以声明为 static。如此依赖，lambda 可以不需要捕获就能访问函数指针，从而实现递归。

▼ 示例

```

1 static unsigned (*fptr)(unsigned);
2
3 static const auto lambda = [] (const unsigned a) {
4     return a < 2 ? a : (*fptr)(a - 2) + (*fptr)(a - 1);
5 };
6
7 static auto init = [] {
8     fptr = +lambda;
9     // Or
10    // fptr = static_cast<LTunsigned (*) (unsigned)>(lambda);
11    return 0;
12 }();
13
14 cout << lambda(10);

```

Lambda 表达式的应用

作为标准库算法的 Predicate（谓词）

从大到小排序：

```

1 std::vector<int> v = {1, 2, 3, 4, 5};
2 std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; });

```

使用 [std::find_if](#) 查找第一个大于 3 的元素：

```

1 std::vector<int> v = {1, 2, 3, 4, 5};
2 auto it = std::find_if(v.begin(), v.end(), [](int a) { return a > 3; });

```

控制中间变量的生命周期

在算法竞赛中，我们会遇到这样的场景：一个变量的初始化需要使用之前声明的变量，其初始化过程又生成占用空间较大的中间变量。

我们希望能尽快析构这些中间变量，以降低内存消耗。此时，我们可以使用 `lambda` 来控制这些中间变量的生命周期。

```

1 void solution(const vector<int>& input) {
2     int b = [&] {
3         vector<int> large_objects(input.size());
4         int c = 0;
5
6         for (int i = 0; i < large_objects.size(); ++i)
7             large_objects[i] = i + input[i];
8
9         for (int i = 0; i < input.size(); ++i) c += large_objects[input[i]];
10
11        return c;
12    }();
13
14    // ...
15 }

```

相较于使用块作用域，`lambda` 可以允许我们使用返回值，使得代码更加简洁；相较于函数，我们不需要额外起名和声明被捕获的各种参数，使得代码更加紧凑。

参考文献

- [cppreference-lambda](#)
 - [Stackoverflow: Overhead with std::function](#)
-

本页面最近更新：2024/12/16 18:11:22, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [c0nstexpr](#), [CCXXXI](#), [CoderOJ](#), [Great-designer](#), [hly1204](#), [huanhuanonly](#), [Persdre](#), [shuzhouliu](#), [Tiphereth-A](#), [zyzzyh](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用